



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS
ICS2121 - MÉTODOS DE OPTIMIZACIÓN - 2025-2

Profesor: JORGE VERA

Ayudantes: - CHRISTOPH BÜRGER (CHRISTOPH.BUERGER@UC.CL)
- FRANCISCO TAMARÍN (FTAMARIN@UC.CL)
- ALBERTO URETA (ALBERTO.URETA@UC.CL)
- MACARENA TORÁN (MACATH@UC.CL)

Taller 13

Técnicas en Programación Entera, Heurísticas, IA y Machine Learning

6 de noviembre

Resumen

Técnicas en Programación Entera

Modelo, relajación y *gaps*:

- (IP) $\min\{c^\top x : Ax \leq b, x \in \mathbb{Z}^n\}$; (LP) $\min\{c^\top x : Ax \leq b\}$.
- Con $S := \{x \in \mathbb{Z}^n : Ax \leq b\}$ y poliedro $P := \{x \in \mathbb{R}^n : Ax \leq b\}$, la *formulación ideal* satisface $P = \text{conv}(S)$.
- Fortaleza: $P_1 \subseteq P_2 \Rightarrow z^* \geq z(P_1) \geq z(P_2)$ (min). *Gap* relativo: $\text{gap} = \frac{UB-LB}{\max\{1, |UB|\}}$, con $LB := \min\{c^\top x : x \in P\}$, $UB := \min\{c^\top x : x \in S\}$.

Integralidad por estructura (unimodularidad):

- A es *totalmente unimodular* (TU) si todo subdeterminante es $0, \pm 1$.
- Si A es TU y b es entero, entonces toda BFS de $\{x : Ax \leq b\}$ es entera $\Rightarrow$ (LP) ya resuelve (IP).
- Ejemplos: matrices de redes bipartitas, flujo en redes, asignación, *matching* bipartito.

Cortes (válidos para S) — esquema general:

- Iterar: resolver (LP) $\rightarrow$ si $x^{\text{LP}} \notin S$, separar una desigualdad válida violada $\alpha^\top x \leq \beta \rightarrow$ agregarla y repetir. Integrado con ramificación: *Branch-and-Cut*.

Familias clave:

- *Covers* (mochila binaria $\sum a_j x_j \leq b$): si C es *cover* ($\sum_{j \in C} a_j > b$),

$$\sum_{j \in C} x_j \leq |C| - 1.$$

Lifting secuencial produce coeficientes fortalecidos; variantes: *flow-covers*, cliques (conflictos 0-1).

Branch-and-Bound (B&B) y Branch-and-Cut (B&C):

0. Raíz: resolver (LP). Inicializar $UB := +\infty$ (o incumbente si existe), $LB := z^{\text{LP}}$, lista de nodos $\mathcal{N}$.
1. Selección de nodo $N \in \mathcal{N}$ (p. ej. *best-bound*, *depth-first*). Resolver su (LP); opcional en B&C: separar cortes (globales/locales) antes de ramificar.
2. Poda por *bound*: si $z_N^{\text{LP}} \geq UB$, descartar N .
3. Poda por infactibilidad: si (LP) de N es infactible, descartar N .
4. Enteralidad: si $x_N^{\text{LP}} \in S$ y $z_N^{\text{LP}} < UB$, actualizar $UB := z_N^{\text{LP}}$ (incumbente).
5. Ramificación (si fraccionario): elegir componente k y crear hijos con $x_k \leq \lfloor x_{N,k}^{\text{LP}} \rfloor$ y $x_k \geq \lceil x_{N,k}^{\text{LP}} \rceil$; actualizar $LB := \min_{M \in \mathcal{N}} z_M^{\text{LP}}$.
6. Parada: si $\mathcal{N} = \emptyset$ o $\frac{UB-LB}{\max\{1, |UB|\}} \leq \varepsilon$, STOP.

Práctica: *strong branching*/pseudocostos para elegir la variable; exploración híbrida; cortes en nodos difíciles; heurísticas primal para bajar UB temprano.

Reformulaciones (fortalecimiento del LP):

- *Formulaciones extendidas*: variables auxiliares y restricciones que modelan estructura (p. ej. flujos, asignaciones) $\Rightarrow$ poliedros más ajustados.
- *Evitar Big-M débiles*: usar M mínimos, restricciones *indicador* o *perspective* para términos convexos activados por binarios.
- *Ruptura de simetrías*: ordenamientos léxicos, desigualdades de dominancia, variables de enlace; mejora bounds y reduce árbol.

Preprocesamiento y heurísticas:

- Fijaciones implícitas, estrechamiento de cotas, eliminación de redundancias, escalamiento, detección rápida de covers/cliques; heurísticas (rondas, *local branching*, *RINS*) para incumbentes.

Criterios de calidad y reporte:

- Mantener $LB \leq z^* \leq UB$ (min), *gap* relativo/absoluto, límites de tiempo/nodos/memoria. En B&C, reportar #cortes, violaciones y mejoras de LB .

Heurísticas

Estructura de una metaheurística:

- Para el problema de optimización general $\min_{x \in C} f(x)$ con C el espacio de soluciones factibles.
- C va a ser nuestro “espacio de búsqueda”.
- Dado $x \in C$ se define, según el problema, la vecindad de x como $V(x) \subset C$.

Procedimiento de búsqueda local:

0. Sea $x^0 \in C, k = 0$.
1. Elegir $x^{k+1} \in V(x^k)$ tal que $f(x^{k+1}) < f(x^k)$.
2. Si se satisface un criterio de parada, STOP.
3. $k = k + 1$, ir a 1.

Simulated Annealing:

0. Sea $x^0 \in C, k = 0$.
1. Elegir al azar $y \in V(x^k)$.
2. Si $f(y) < f(x^k)$, entonces $x^{k+1} = y, k = k + 1$, ir a 1.
3. Si $f(y) \geq f(x^k)$, entonces aceptar y con probabilidad:

$$p = e^{-\frac{f(y) - f(x^k)}{BT_k}}$$

4. Si se satisface un criterio de parada, STOP.
5. $x^{k+1} = y, k = k + 1$, ir a 1.
6. Si y no es aceptado, ir a 1.

Donde B una constante positiva y $T_{i+1} = \alpha T_i$, con $\alpha < 1$ una sucesión decreciente que converge a 0 cuando $i \rightarrow \infty$.

Búsqueda Tabú:

0. Sea $x^0 \in C, k = 0, L = \{\}$.
1. Etapa k : Estamos en x^k y se verifica criterio de parada.
2. Si se continúa, construir vecindad $V(x^k)$.
3. Elegir y solución de $\min_{x \in V(x^k)} f(x)$.

-
4. Si $y \in L$, elegir otro y tal que $f(y) < f(x^k)$, $y \notin L$.
 5. Si no se puede encontrar y adecuado, hacer uno de los siguientes:
 - 5.1. Aceptar y tal que $f(y) > f(x^k)$ con alguna probabilidad.
 - 5.2. Aplicar DIVERSIFICACIÓN para generar un nuevo y .
 6. $x^{k+1} = y$, $k = k + 1$, $L = L \cup \{y\}$, ir a 1 .

Ejercicio 1

Una empresa posee un conjunto de bodegas entre las cuales se generan necesidades de transporte. Esto se puede representar por una red con un conjunto de nodos, N , que corresponden a las bodegas, y un conjunto de arcos, A , formado por pares (i, j) que representan la posibilidad de viajar entre la bodega i y la j . Existe una oferta/demanda d_i en cada nodo $i \in N$ (la convención habitual, si $d_i > 0$ es oferta, si es < 0 es demanda y si es $= 0$ es un nodo de paso), y existe un costo variables c_{ij} por el transporte entre i y j . Sin embargo, también existe un costo fijo, F_{ij} por efectuar transporte entre i y j . Adicionalmente, si se hace uso de la posibilidad de transportar entre i y j , se produce una externalidad ambiental negativa igual a β_{ij} (en unidades adecuadas) y la externalidad total no puede superar un valor B . Además, la cantidad que se puede transportar por (i, j) no puede ser mayor a u_{ij} . La empresa quiere determinar de qué forma realizar el transporte y para eso formula un modelo de optimización con variables x_{ij} que corresponden a la cantidad que se transporte entre i y j y variables binarias z_{ij} que indican si se usa o no la posibilidad de transportar entre i y j . El modelo es el siguiente, que corresponde a un modelo de “diseño de red” con restricciones adicionales:

$$\begin{aligned} \text{mín} \quad & \sum_{(i,j) \in A} c_{ij}x_{ij} + \sum_{(i,j) \in A} F_{ij}z_{ij} \\ \text{s. a.} \quad & \sum_{j:(i,j) \in A} x_{ij} - \sum_{k:(k,i) \in A} x_{ki} = d_i \quad i \in N \end{aligned} \quad (1)$$

$$x_{ij} \leq u_{ij}z_{ij} \quad (i, j) \in A \quad (2)$$

$$\sum_{(i,j) \in A} \beta_{ij}z_{ij} \leq B \quad (3)$$

$$x_{ij} \geq 0, z_{ij} \in \{0, 1\} \quad (i, j) \in A, i \in N \quad (4)$$

Se propone abordar el problema mediante una heurística tipo Simulated Annealing. Indique cuál sería su “espacio de búsqueda” para realizar el Simulated Annealing y cómo construiría un “vecino” válido de la solución actual. Además, escriba todos los pasos de la heurística.

Solución:

En este caso, el espacio de búsqueda puede tomarse como el definido por las variables binarias z , que será discreto. Entonces, tendremos que una solución cualquiera está dada por un caso particular del vector $z = (z_{ij})_{ij}$. El espacio de búsqueda asociado sería entonces:

$$Z = \left\{ z \in \{0, 1\}^{|A|} : \sum_{(i,j) \in A} \beta_{ij}z_{ij} \leq B \right\}$$

Luego podemos definir las funciones:

$f, g : Z \longrightarrow \mathbb{R}$ como:

$$\begin{aligned}
 g(z) &= \min \sum_{(i,j)} c_{ij} x_{ij} \\
 \text{s.a} \quad & \sum_{j:(i,j) \in A} x_{ij} - \sum_{k:(k,i) \in A} x_{ik} = d_i \quad i \in N \quad (\pi_i) \\
 & x_{ij} \leq u_{ij} z_{ij} \quad (i,j) \in A \quad (\theta_{ij}) \\
 & x_{ij} \geq 0 \quad (i,j) \in A, i \in N
 \end{aligned}$$

$$f(\hat{z}) = \sum_{(i,j) \in A} F_{ij} \hat{z}_{ij} + g(z)$$

donde f será la función que utilizaremos para comparar vecinos.

Nótese que el problema de optimización tiene fijos los valores de $\hat{z}$, y se trata de un problema de optimización lineal y debería ser muy fácil de resolver.

Entonces, el problema que tratará de resolver la heurística es:

$$\begin{aligned}
 \text{mín} \quad & f(z) \\
 \text{s. a.} \quad & z \in Z
 \end{aligned}$$

Veamos ahora cómo elegir vecinos. Supongamos que estamos en una solución $\hat{z}$. Entonces, una solución vecina puede construirse con algunas de las siguientes opciones (existen muchas más opciones): Sea $\hat{z} \in Z$ una solución factible actual. Definimos su vecindad $V(\hat{z})$ como el conjunto de soluciones factibles obtenidas mediante modificaciones elementales sobre $\hat{z}$. Un procedimiento válido es:

Algorithm 1 Generación de vecindad $V(\hat{z})$

Require: Solución actual $\hat{z} \in Z$, número de vecinos a generar $N \in \mathbb{N}$

Ensure: Vecindad $V(\hat{z}) \subseteq Z$

- 1: Inicializar $V(\hat{z}) \leftarrow \emptyset$
 - 2: **while** $|V(\hat{z})| < N$ **do**
 - 3: Realizar este subprocedimiento
 - (a) Tomar una arista (i, j) cualquiera e invertir su valor, es decir hacer la operación $z_{ij} \leftarrow 1 - z_{i,j}$
 - (b) Si esa asignación es factible añadirla a V , si no es factible tomamos aristas (i', j') tal que $z_{i'j'} = 1$ al azar y cambiar su valor a 0 hasta que z cumpla con la restricción (3)
 - 4: **if** $z' \in Z$ **then**
 - 5: Agregar z' a $V(\hat{z})$
 - 6: **end if**
 - 7: **end while**
 - 8: **return** $V(\hat{z})$
-

Puede haber otras opciones de modificaciones locales de una solución para construir una vecindad $V(\hat{z})$ de una solución $\hat{z}$.

Con estos elementos se puede hacer el Summulated Annealing habitual.

0. Sea $z^0 \in Z$.
1. Armar $V(\hat{z}^k)$.
2. Elegir al azar $y \in V(\hat{z}^k)$.
3. Si $g(y) < g(z^k)$, entonces $z^{k+1} = y, k = k + 1$.
4. Si $g(y) \geq g(\hat{z}^k)$, entonces aceptar y con probabilidad

$$p = e^{-\frac{g(\hat{z}^k) - g(y)}{BT_k}}$$

- 4.1. Si y es aceptado, entonces $\hat{z}^{k+1} = y, k = k + 1$.
- 4.2. Si y no es aceptado, entonces $\hat{z}^{k+1} = \hat{z}^k, k = k + 1$.
5. Verificar criterio de parada y STOP si se cumple. Si no, ir a 1.

Otra opción es simplemente realizar el procedimiento una sola vez (generar un vecino). Esto quedaría como, sea $z \in Z$ podemos construir un vecino tomando una arista (i, j) cualquiera e invertir su valor, es decir hacer la operación $z_{ij} \leftarrow 1 - z_{i,j}$ si es que el resultado de esta operación no pertenece a Z , entonces tomamos aristas (i', j') tal que $z_{i'j'} = 1$ al azar y cambiar su valor a 0 hasta que z cumpla con la restricción (3). Luego:

-
0. Sea $z^0 \in Z$.
 2. Elegir vecino y .
 3. Si $g(y) < g(z^k)$, entonces $z^{k+1} = y, k = k + 1$.
 4. Si $g(y) \geq g(\hat{z}^k)$, entonces aceptar y con probabilidad

$$p = e^{-\frac{g(\hat{z}^k) - g(y)}{BT_k}}$$

- 4.1. Si y es aceptado, entonces $\hat{z}^{k+1} = y, k = k + 1$.
- 4.2. Si y no es aceptado, entonces $\hat{z}^{k+1} = \hat{z}^k, k = k + 1$.
5. Verificar criterio de parada y STOP si se cumple. Si no, ir a 1.

Ejercicio 2

Considere la siguiente conversación con DeepSeek en su versión DeepThink:

Nosotros:

Hola! continuación te voy a entregar un modelo matemático de optimización:

Vamos a considerar la situación de una compañía de transportes del tipo “ridesharing” (como las que operan a través de plataformas de Internet), la que quiere planificar la cantidad de conductores que debe tener en cada zona para satisfacer la demanda. Vamos a suponer que hay n zonas y existe una demanda estimada de viajes igual a d_j^t en la zona j en el periodo t . Los conductores (en sus autos) operan en ciertas zonas pero pueden ser movidos de una zona a otra (reposicionamiento, se llama esto) y esto tiene un costo de c_{ij}^t por conductor que se mueve entre la zona i y la zona j en el periodo t . Existe un beneficio α_j^t por conductor, por cubrir viajes en la zona j en el periodo t . Además, cada vez que se reposicionan conductores desde la zona j , se incurre en un costo fijo igual a f_j^t en el periodo t , independientemente de la cantidad de conductores reposicionados e independientemente de su destino. Suponemos que cada conductor atiende un solo viaje. El contrato con los conductores de los autos especifica ciertos límites a la cantidad de reposicionamientos que se pueden hacer por periodo de tiempo, siendo igual a G^t para el periodo t . Igualmente, en cada periodo existen ciertas zonas en las que no puede haber reposicionamiento de conductores si los hay en otra. Esto se describe con un conjunto E_j^t para la zona j en el periodo t .

La empresa quiere determinar las cantidades de conductores a reposicionar entre las zonas de modo de maximizar la utilidad neta y para esto, ha modelado este problema usando las siguientes variables:

x_{ij}^t : cantidad de conductores reposicionados desde la zona i a la zona j en el periodo t .

u_j^t : cantidad de conductores disponibles en la zona j en el periodo t .

z_j^t : variable binaria que vale 1 si se reposicionan conductores de la zona j en t .

Para simplificarse la vida, y dado que la cantidad de conductores es relativamente grande, la empresa decide considerar las variables x y las u como continuas en vez de enteras (como deberían ser).

El modelo empleado es el siguiente:

$$\begin{aligned}
\text{mín} \quad & \sum_{t=1}^T \sum_{j=1}^n \sum_{i=1, i \neq j}^n c_{ij}^t x_{ij}^t + \sum_{t=1}^T \sum_{j=1}^n f_j^t z_j^t - \sum_{t=1}^T \sum_{j=1}^n \alpha_j^t u_j^t \\
\text{s. a.} \quad & u_j^t = u_j^{t-1} + \sum_{i=1, i \neq j}^n x_{ij}^t - \sum_{i=1, i \neq j}^n x_{ji}^t & j = 1, \dots, n; t = 1, \dots, T \quad (1) \\
& u_j^t \leq d_j^t & j = 1, \dots, n; t = 1, \dots, T \quad (2) \\
& \sum_{i=1, i \neq j}^n x_{ji}^t \leq M z_j^t & j = 1, \dots, n; t = 1, \dots, T \quad (3) \\
& \sum_{j=1}^n z_j^t \leq G^t & t = 1, \dots, T \quad (4) \\
& \sum_{i \in E_j^t} z_i^t \leq n(1 - z_j^t) & j = 1, \dots, n; t = 1, \dots, T \quad (5) \\
& z_j^t \in \{0, 1\}, x_{ij}^t \geq 0, u_j^t \geq 0 & j = 1, \dots, n; t = 1, \dots, T \quad (6)
\end{aligned}$$

donde las cantidades u_j^0 corresponden al número de conductores iniciales en cada zona y los asumimos conocidos (no son igual a cero, si no, no habría conductores para atender a los pasajeros). El número M es una constante “suficientemente grande”.

Las restricciones (1) hacen el balance de conductores disponibles en una zona, considerando los que han llegado de otras zonas y los que van a otras zonas (suponemos que los reposicionamientos se hacen al comienzo del periodo); las restricciones (2) establecen que el número de conductores en la zona no debe ser mayor a la demanda (notar que no necesariamente cumplimos demanda, todo es un tema de maximizar utilidades); las (3) activan las variables binarias si hay reposicionamientos a otras zonas; las (4) imponen el límite de veces que se pueden hacer reposicionamientos y, las (5), establecen que si la zona j hace reposicionamientos, las zonas en el conjunto E_j^t no pueden. Finalmente, (6) establece la naturaleza de las variables.

Deepseek

Modelo de Optimización para la Planificación de Conductores en Ridesharing

A continuación se presenta el modelo matemático de optimización formulado en LaTeX, que busca maximizar la utilidad neta de la compañía de transporte considerando reposicionamiento de conductores, costos fijos, beneficios por viaje y restricciones operativas.

$$\begin{aligned}
\text{mín} \quad & \sum_{t=1}^T \sum_{j=1}^n \sum_{\substack{i=1 \\ i \neq j}}^n c_{ij}^t x_{ij}^t + \sum_{t=1}^T \sum_{j=1}^n f_j^t z_j^t - \sum_{t=1}^T \sum_{j=1}^n \alpha_j^t u_j^t \\
\text{s.a.} \quad & u_j^t = u_j^{t-1} + \sum_{\substack{i=1 \\ i \neq j}}^n x_{ij}^t - \sum_{\substack{i=1 \\ i \neq j}}^n x_{ji}^t \quad \forall j = 1, \dots, n; \quad t = 1, \dots, T \quad (1) \\
& u_j^t \leq d_j^t \quad \forall j = 1, \dots, n; \quad t = 1, \dots, T \quad (2) \\
& \sum_{\substack{i=1 \\ i \neq j}}^n x_{ji}^t \leq M z_j^t \quad \forall j = 1, \dots, n; \quad t = 1, \dots, T \quad (3) \\
& \sum_{j=1}^n z_j^t \leq G^t \quad \forall t = 1, \dots, T \quad (4) \\
& \sum_{i \in E_j^t} z_i^t \leq n(1 - z_j^t) \quad \forall j = 1, \dots, n; \quad t = 1, \dots, T \quad (5) \\
& z_j^t \in \{0, 1\}, \quad x_{ij}^t \geq 0, \quad u_j^t \geq 0 \quad \forall i, j = 1, \dots, n; \quad t = 1, \dots, T \quad (6)
\end{aligned}$$

Explicación Breve del Modelo:

- Función Objetivo: Minimiza el costo total neto (costos de reposicionamiento y fijos menos beneficios por viajes atendidos).
- Restricciones:
 - Balance de conductores en cada zona y periodo.
 - Límite superior de conductores respecto a la demanda.
 - Activación de variables binarias z_j^t cuando hay reposicionamiento.
 - Límite máximo de reposicionamientos por periodo.
 - Exclusividad en reposicionamiento: si la zona j reposiciona, las zonas en E_j^t no pueden hacerlo.
 - Naturaleza de las variables.

Este modelo permite a la empresa optimizar la asignación dinámica de conductores entre zonas, maximizando su utilidad neta bajo las restricciones operativas.

Nosotros:

Se puede resolver con relajación lagrangiana? Si es así, desarrolla la descomposición

DeepSeek:

Sí, el modelo se puede resolver mediante relajación lagrangiana, descomponiéndolo en subproblemas más manejables. La idea central es relajar las restricciones que acoplan las variables, introduciendo multiplicadores de Lagrange, para luego resolver un problema relajado que se separa por periodos y zonas. A continuación, se detalla el desarrollo paso a paso.

Relajación Lagrangiana

Se relajan las restricciones (1), (4) y (5), ya que son las que acoplan las variables entre zonas y periodos. Se introducen los siguientes multiplicadores de Lagrange:

- λ_t^j para las restricciones (1) (de igualdad, sin restricción de signo)
- $\mu_t \geq 0$ para las restricciones (4).
- $v_t \geq 0$ para las restricciones (5).

La función lagrangiana resultante es:

$$\begin{aligned}
L = & \sum_{t=1}^T \sum_{j=1}^n \sum_{\substack{i=1 \\ i \neq j}}^n (c_{ij}^t + \lambda_j^t - \lambda_i^t) x_{ij}^t \\
& + \sum_{t=1}^T \sum_{j=1}^n \left(f_j^t + \mu_t + \sum_{k:j \in E_k^t} \nu_k^t + n \nu_j^t \right) z_j^t \\
& + \sum_{t=1}^T \sum_{j=1}^n (-\alpha_j^t + \lambda_j^t - \lambda_j^{t+1}) u_j^t \\
& - \sum_{t=1}^T \mu_t G^t - \sum_{t=1}^T \sum_{j=1}^n n \nu_j^t + \text{términos constantes con } u_j^0
\end{aligned}$$

donde $\lambda_j^{T+1} = 0$ para todo j .

Problema Relajado y Descomposición El problema relajado consiste en minimizar L sobre x, u, z sujeto solo a las restricciones (2), (3) y (6). Este problema se descompone por periodo t y por zona j , ya que la función objetivo y las restricciones restantes son separables. Para cada t y j , las variables se determinan de la siguiente manera:

Para u_j^t :

Sea $c_j^t = -\alpha_j^t + \lambda_j^t - \lambda_j^{t+1}$. Entonces:

$$u_j^t = \begin{cases} d_j^t & \text{si } c_j^t < 0 \\ 0 & \text{en caso contrario} \end{cases}$$

Para z_i^t y x_{ij}^t (para cada i):

- Sea $a_{ij}^t = c_{ij}^t + \lambda_j^t - \lambda_i^t$ para cada $j \neq i$.
- Calcule $m_i^t = \min_{j \neq i} a_{ij}^t$.
- Sea $b_i^t = f_i^t + \mu_t + \sum_{k: i \in E_k^t} \nu_k^t + n\nu_i^t$.

Entonces:

- Si $b_i^t + M \cdot m_i^t < 0$, entonces $z_i^t = 1$ y se elige j^* tal que $a_{ij^*}^t = m_i^t$. Se define $x_{ij^*}^t = M$ y $x_{ik}^t = 0$ para $k \neq j^*$.
- En caso contrario, $z_i^t = 0$ y $x_{ij}^t = 0$ para todo $j \neq i$.

Optimización Dual

El dual lagrangiano se define como:

$$\max \theta(\lambda, \mu, v)$$

Donde $\max \theta(\lambda, \mu, v) = \min L(x, u, z, \lambda, \mu, v)$

Para resolver el dual, se puede utilizar un método de subgradiente. Los subgradientes para las restricciones relajadas son:

- Para (1): $u_j^t - u_j^{t-1} - \sum_{i \neq j} x_{ij}^t + \sum_{i \neq j} x_{ji}^t$ para cada j, t .
- Para (4): $\sum_j z_j^t - G^t$ para cada t .
- Para (5): $\sum_{i \in E_j^t} z_i^t - n(1 - z_j^t)$ para cada j, t .

Conclusión La relajación lagrangiana permite descomponer el problema original en subproblemas independientes por periodo y zona, que son fáciles de resolver. El método iterativo de actualización de multiplicadores mediante subgradiente converge hacia la solución dual, proporcionando una cota para el problema primal y, en muchos casos, una solución factible cercana al óptimo.

- (a) Respecto a la solución propuesta por Deepseek, comente si se cumple con el objetivo de la pregunta.

Solución: Sí se cumple, ya que se desarrolla una descomposición mediante una heurística de relajación lagrangiana. Cumple con todos los “pasos típicos”:

- Reconocer restricciones complicantes
- Reestructurar modelo
- Separar el modelo
- Encontrar el dual a resolver
- Definir el subgradiente

Sin embargo, que se cumpla con el objetivo no significa que lo hizo de la mejor manera.

- (b) Este mismo ejercicio se desarrolló en el taller 11, pregunta 1. Comente sobre las

diferencias en ambas soluciones.

Solución: Lo primero que se debería notar es que en la solución “oficial” se relaja solo la restricción (3), lo que lleva a un procedimiento completamente diferente.

En primer lugar, relajar (3) simplifica la estructura del modelo y mantiene la coherencia de flujo (R1), el límite global (R4) y las exclusiones (R5).

Por eso esta relajación es natural y efectiva: reduce el acoplamiento binario–continuo, que es la fuente principal de dificultad en el problema.

Por otro lado, la restricción 1 asegura conservación de conductores (lo que entra menos lo que sale). Si la relajamos, estamos permitiendo crear o destruir conductores artificialmente, lo que rompe la interpretación del modelo.

La relajación Lagrangiana aquí sería muy poco útil desde el punto de vista operativo, porque los subproblemas dejarían de tener sentido físico (ya no sabríamos de dónde vienen los conductores), y el valor dual resultante no daría buena cota inferior (en el caso de minimización) o superior (en maximización).

Además, la restricción 4 controla cuántas zonas pueden reposicionar conductores en total por período. Relajarla implica que el modelo puede activar más zonas de las permitidas, pagando un “precio” μ_t por exceder el límite.

Esto tiene cierto sentido, pero el subproblema no se simplifica tanto como al relajar (3), y además, la restricción (4) solo acopla las variables binarias entre sí, no a las continuas, por lo que el problema discreto sigue siendo difícil.

Finalmente, la restricción 5 son de exclusión lógica: si una zona j hace reposicionamiento, las zonas en E_j^t no pueden hacerlo.

Relajarlas elimina dependencias entre las decisiones binarias de distintas zonas, permitiendo resolver independientemente por zona.

Eso puede ser útil si el conjunto es lo que genera la complejidad combinatoria principal. Sin embargo, las z_{jt} seguirán vinculadas a los x_{ijt} por (3), por lo que relajar (5) ayuda con las restricciones lógicas entre zonas, pero no ataca la fuente de acoplamiento entre decisiones continuas y binarias.

Así, relajar (1), (4) y (5) pierde sentido práctico y debilita mucho el modelo.

Ejercicios Propuestos

Ejercicio 1

Considere un problema lineal entero que incluye variables x_i , $i = 1, \dots, n$, y la restricción:

$$\sum_{i=1}^n x_i = 1, \quad x_i \in \{0, 1\}.$$

- (a) Considere la regla de *branching* usual donde, dada una solución x con $x_j \notin \{0, 1\}$, se generan dos nodos con las restricciones $x_j = 0$ y $x_j = 1$, respectivamente. Explique por qué esta regla podría generar un árbol de *branch-and-bound* desbalanceado.

Solución: En la rama dada por $x_j = 0$, hay $n - 1$ variables x_i que podrían tomar el valor 1. Por otro lado, en la rama dada por $x_j = 1$, las variables x_i restantes quedan fijadas en 0. Luego, en la primera rama se podrían generar muchos más nodos que en la segunda, lo que produce un árbol desbalanceado.

- (b) Considere una regla de *branching* donde, dada una solución x fraccionaria y un conjunto $I \subseteq \{1, \dots, n\}$ tal que $x_i \notin \{0, 1\}$ para $i \in I$, se generan dos nodos con las restricciones:

$$\sum_{i \in I} x_i = 0 \quad \text{y} \quad \sum_{i \in I} x_i = 1,$$

respectivamente. Explique por qué esta regla es correcta y por qué podría ayudar a mantener el balance del árbol.

Solución: Primero, notemos que esta regla elimina la solución fraccionaria x , pero mantiene todas las soluciones enteras del problema, por lo que es correcta. Ahora, si el conjunto I particiona $\{1, \dots, n\}$ de forma relativamente equitativa, entonces la cantidad de nodos que pueden ser generados en ambas ramas debería ser más similar que con la regla usual, ayudando así a mantener el árbol más balanceado.

Ejercicio 2

- (a) Usted está ayudando a la universidad en la asignación de salas para las interrogaciones. Para un día dado del semestre, existe un conjunto S de salas disponibles y un conjunto C de cursos que rendirán una interrogación. El curso $c \in C$ cuenta con v_c vacantes, mientras que la sala $s \in S$ cuenta con capacidad para k_s estudiantes. Cada sala no puede ser asignada a más de un curso, y las capacidades de las salas asignadas a un curso dado deben al menos duplicar las vacantes del mismo. Además, cada curso $c \in C$ puede dividirse en a lo más p_c salas. Idealmente, las salas asignadas a un mismo curso deben estar cerca entre ellas. La distancia entre las salas $s_1, s_2 \in S$ es d_{s_1, s_2} .

Formule un problema lineal entero que le permita minimizar la suma de las distancias de las salas asignadas a cada curso.

Solución: Usamos variables binarias x_{cs} que toman el valor 1 si y solo si la sala s es asignada al curso c . Definimos A como el conjunto de pares de salas diferentes, y usamos variables binarias y_{ca} que toman el valor 1 si y solo si las salas del par a son asignadas al curso c .

$$\begin{aligned}
 \text{mín} \quad & \sum_{c \in C} \sum_{a \in A} d_a y_{ca} \\
 \text{s.a.} \quad & \sum_{s \in S} x_{cs} \leq p_c, & \forall c \in C, \\
 & \sum_{s \in S} k_s x_{cs} \geq 2v_c, & \forall c \in C, \\
 & \sum_{c \in C} x_{cs} \leq 1, & \forall s \in S, \\
 & x_{cs_1} + x_{cs_2} \leq y_{ca} + 1, & \forall c \in C, a = (s_1, s_2) \in A, \\
 & x_{cs} \in \{0, 1\}, & \forall (c, s) \in C \times S, \\
 & y_{ca} \in \{0, 1\}, & \forall (c, a) \in C \times A.
 \end{aligned}$$

- (b) Dadas variables $x_1, x_2, z \in \{0, 1\}$, indique una formulación lineal para la restricción:

$$z = 1 \iff (x_1 = 1 \wedge x_2 = 0) \vee (x_1 = 0 \wedge x_2 = 1).$$

Solución: Notemos que:

$$z = 1 \iff x_1 \neq x_2 \iff |x_1 - x_2| = 1.$$

Usamos entonces las siguientes desigualdades lineales:

$$\begin{aligned}
x_1 - x_2 &\leq z, \\
-x_1 + x_2 &\leq z, \\
z &\leq x_1 + x_2, \\
z &\leq 2 - x_1 - x_2.
\end{aligned}$$

(c) Dado $X \subseteq \mathbb{R}^n$, sea

$$(P) \quad \min\{c^\top x : Ax \leq b, x \in X\}.$$

Dado $\mu \geq 0$, sea

$$(R) \quad \min\{c^\top x + \mu^\top (Ax - b) : Ax \leq b, x \in X\}.$$

Demuestre que (R) es una relajación de (P) .

Solución: Como ambos problemas tienen el mismo conjunto factible, basta ver que:

$$c^\top x \leq c^\top x + \mu^\top (Ax - b), \quad \forall x \text{ factible.}$$

En efecto, todo x factible satisface $Ax \leq b$, es decir, $Ax - b \leq 0$. Como $\mu \geq 0$, se tiene que $\mu^\top (Ax - b) \leq 0$, lo que implica que:

$$c^\top x \leq c^\top x + \mu^\top (Ax - b),$$

y, por tanto, (R) es una relajación de (P) .

Ejercicio 3

Considere una heurística GRASP. A continuación se le presenta la evolución, a través de las iteraciones, de la solución actual para una instancia particular, que considera la inversión y tiempo de viaje de la solución:

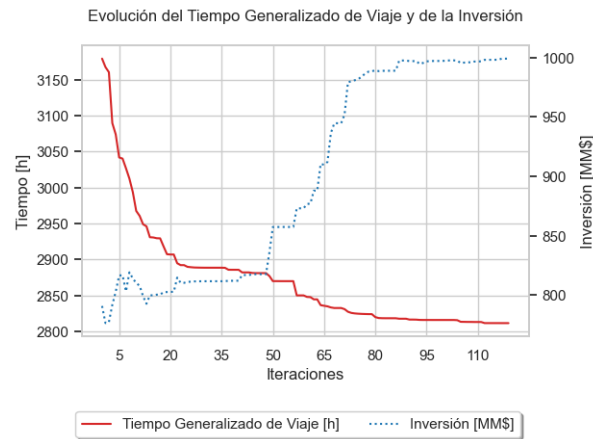


Figura 1: Evolución del Tiempo Generalizado de Viaje y de la Inversión

Como podrá notar, el valor de la función objetivo es no creciente a lo largo de las iteraciones. Responda las siguientes preguntas:

- (a) ¿A qué se debe dicho comportamiento?

Solución: GRASP realiza exclusivamente movimientos de descenso *greedy*.

- (b) Usted propone utilizar Simulated Annealing para este problema. ¿Qué cambio esperaría respecto a la curva 'Tiempo Generalizado de Viaje'? ¿Siguiendo siendo monótona decreciente? Justifique con precisión.

Solución: La curva ya no tiene garantía de ser monótona decreciente, ya que SA puede aceptar (en probabilidad) soluciones con 'peor' función objetivo.

- (c) Muchas heurísticas dependen de parámetros que permiten transitar de la exploración del dominio, a la explotación de soluciones. ¿Es recomendable usar siempre los mismos valores para los parámetros en todas las instancias de un problema? Justifique con precisión. En caso de ser positiva su respuesta, indique por qué aquello es así. En caso de ser negativa su respuesta, indique qué es lo que se debe hacer.

Solución: No es recomendable. Por ejemplo, en el caso de SA, la probabilidad de aceptación de soluciones 'malas' depende de la magnitud de la función objetivo, por lo que es necesario reconfigurar parámetros como la temperatura, la constante B , entre otros. Alternativas de calibración hay muchas, pero, como mínimo, es de esperar un experimento de 'ensayo y error' hasta tener un comportamiento deseable.

Ejercicio 4

Considere un problema de Optimización de la siguiente forma:

$$\begin{aligned} \text{mín} \quad & c^T x \\ \text{s.a.} \quad & Ax \leq b \\ & Dx \leq e \\ & x \text{ entero} \end{aligned}$$

(Todas las matrices y vectores tienen dimensiones consistentes.) Suponga que cualquiera de los dos conjuntos de restricciones ($Ax \leq b$ o $Dx \leq e$) puede calificar como “complicante” y juntas hacen el problema muy difícil, aunque por separado son más abordables. En ese caso, se puede hacer lo siguiente: se definen nuevas variables y , y se escribe el siguiente problema:

$$\begin{aligned} \text{mín} \quad & \frac{1}{2}(c^T x + c^T y) \\ \text{s.a.} \quad & Ax \leq b \\ & Dy \leq e \\ & x - y = 0 \\ & x, y \text{ enteros} \end{aligned}$$

De este modo, las y son “copias” de las variables x . Aprovechando esta nueva estructura, proponga una relajación lagrangeana para este último problema y explique qué ventaja tiene esa relajación propuesta por usted. Debe especificar la función dual y cómo se resuelve el correspondiente problema relajado, qué subgradiente usaría, etc. (Este enfoque se conoce como Descomposición Lagrangeana).

Solución: El Lagrangeano del problema es:

$$L(x, \lambda) = \frac{1}{2}(c^T x + c^T y) + \lambda^T(x - y)$$

y la correspondiente función dual es:

$$\begin{aligned} \theta(\lambda) = \text{mín} \quad & \frac{1}{2}(c^T x + c^T y) + \lambda^T(x - y) \\ \text{s.a.} \quad & Ax \leq b \\ & Dy \leq e \\ & x, y \text{ enteros} \end{aligned}$$

que equivale a:

$$\begin{aligned} \theta(\lambda) = \text{mín} \quad & \left(\frac{1}{2}c + \lambda\right)^T x + \left(\frac{1}{2}c - \lambda\right)^T y \\ \text{s.a.} \quad & Ax \leq b \\ & Dy \leq e \\ & x, y \text{ enteros} \end{aligned}$$

Este problema se separa en dos, $\theta(\lambda) = \theta_1(\lambda) + \theta_2(\lambda)$, donde

$$\begin{aligned} \theta_1(\lambda) = \min \quad & \left(\frac{1}{2}c + \lambda \right)^T x \\ \text{s.a.} \quad & Ax \leq b \\ & x \text{ entero} \end{aligned}$$

$$\begin{aligned} \theta_2(\lambda) = \min \quad & \left(\frac{1}{2}c - \lambda \right)^T y \\ \text{s.a.} \quad & Dy \leq e \\ & y \text{ entero} \end{aligned}$$

Cada uno de estos se puede resolver por separado, recuerde que se dice que son “manejables”. Ahora, para resolver el dual, $\max_{\lambda \in \mathbb{R}^n} \theta(\lambda)$, se aplica el método de “subgradientes”. Si estamos en una iteración k y hemos obtenido soluciones x^k e y^k en los dos subproblemas, un subgradiente es

$$h^k = x^k - y^k.$$

Este se usa para construir el nuevo punto,

$$\lambda^{k+1} = \lambda^k + th^k,$$

donde t es un paso determinado en forma adecuada.

Ejercicio 5

Consideraremos en esta pregunta la situación de una cooperativa formada por familias de pequeños agricultores los cuales deben planificar sus cultivos con el objetivo de tener suficientes productos cultivados para su propio sustento, y también poder venderlo en el mercado y obtener ingresos. La cooperativa quiere planificar los cultivos de cada familia y también da la posibilidad de que estas familias intercambien cultivos.

Más en concreto, hay m familias de agricultores, cada una de las cuales puede plantar n cultivos diferentes. La familia i dispone de una superficie de S_i [m^2]. Cada cultivo tiene un rendimiento específico, el cultivo j rinde r_j kilos de producto final por m^2 sembrado. El costo de siembra del cultivo j es c_j por m^2 y hay un costo fijo F_j si se siembra del cultivo j . Por otro lado, el costo de cosecha es d_j por m^2 , para el cultivo j . El producto cosechado del cultivo j se puede vender en el mercado a p_j por kilo cosechado. Por otro lado, la familia i debe procurar tener una cantidad mínima igual a f_{ij} del cultivo j después de cosechar. Esto es para garantizar su propio sustento. Los agricultores pueden intercambiar productos cosechados entre ellos, de modo que eso les ayuda a cumplir con los mínimos especificados, pero sólo venden de la cosecha propia. Adicionalmente, por razones de salud del suelo, para cada cultivo j , existe un conjunto de otros cultivos, que denotaremos R_j que son “incompatibles” con j , es decir, si se siembra j , no puede sembrarse ninguno de los otros cultivos identificados en el conjunto R_j .

La cooperativa ha construido el siguiente modelo de optimización para determinar su política de siembra de modo de cumplir con los mínimos de cada familia y, además, ganar la mayor cantidad de dinero posible. En este modelo, x_{ij} corresponde a la superficie sembrada del producto j por la familia i , z_{ij} es una variable binaria que indica si se sembró del cultivo j por la familia i o no, y_{ilj} indica la cantidad de cultivo j que la familia i le traspasa a la familia l . También se usa la variable v_{ij} que representa la cantidad del cultivo j vendida en el mercado por la familia i , y las variables w_{ij} que indican la cantidad cosechada del cultivo j que la familia i conserva para sí misma:

$$\begin{aligned}
 & \text{máx} \quad \sum_{i=1}^m \sum_{j=1}^n p_j v_{ij} - \sum_{i=1}^m \sum_{j=1}^n \{c_j x_{ij} + d_j x_{ij} + F_j z_{ij}\} \\
 & \text{s.a.} \\
 & \quad r_j x_{ij} = v_{ij} + w_{ij} + \sum_{l=1, l \neq i}^m y_{ilj} \quad i = 1, \dots, m; j = 1, \dots, n \quad (1) \\
 & \quad \sum_{j=1}^n x_{ij} \leq S_i \quad i = 1, \dots, m \quad (2) \\
 & \quad w_{ij} + \sum_{l=1, l \neq i}^m y_{lij} \geq f_{ij} \quad i = 1, \dots, m; j = 1, \dots, n \quad (3) \\
 & \quad x_{ij} \leq S_i z_{ij} \quad i = 1, \dots, m; j = 1, \dots, n \quad (4) \\
 & \quad \sum_{k \in R_j} z_{ik} \leq n(1 - z_{ij}) \quad i = 1, \dots, m; j = 1, \dots, n \quad (5) \\
 & \quad x_{ij} \geq 0, y_{ilj} \geq 0, v_{ij} \geq 0, w_{ij} \geq 0, z_{ij} \in \{0, 1\} \quad i, l = 1, \dots, m; j = 1, \dots, n \quad (6)
 \end{aligned}$$

La función objetivo hace el balance de utilidades para la cooperativa completa, las restricciones (1) establecen la división de lo cosechado entre lo que se vende, lo que se queda en la familia y lo que se envía a otros, las (2) establecen el límite de superficie a sembrar, las (3) hacen el balance para cumplir con los mínimos, las (4) activan las variables binarias para el costo fijo, las (5) establecen las exclusiones de cultivos. Finalmente, (6) define la naturaleza de las variables.

- a) Identifique alguna estructura de restricciones complicantes y plantee una Relajación Lagrangiana basada en las restricciones complicantes que usted seleccione (puede ser uno o varios conjuntos de restricciones, es su elección). Justifique su elección. Sea claro en el problema relajado de la función dual, explicando qué tipo de problema resulta y cómo debería resolverse. Justifique.

Solución: Para este inciso pueden haber varias respuestas correctas, se podría por ejemplo relajar (4), (5), (4) y (5). (4) ya que relaciona variables binarias con continuas, y (5) ya que también hay una relación entre variables. Para efectos de pauta se relajará (4), así nos queda

$$\begin{aligned}
 \theta(\lambda)_{\lambda \geq 0} = \max \quad & \sum_{i=1}^m \sum_{j=1}^n p_j v_{ij} - \sum_{i=1}^m \sum_{j=1}^n \{c_j x_{ij} + d_j x_{ij} + F_j z_{ij}\} + \sum_{i=1}^m \sum_{j=1}^n \lambda_{ij} (S_i z_{ij} - x_{ij}) \\
 \text{s.a. } & r_j x_{ij} = v_{ij} + w_{ij} + \sum_{l=1, l \neq i}^m y_{ilj} \quad i = 1, \dots, m; j = 1, \dots, n \\
 & \sum_{j=1}^n x_{ij} \leq S_i \quad i = 1, \dots, m \\
 & w_{ij} + \sum_{l=1, l \neq i}^m y_{lij} \geq f_{ij} \quad i = 1, \dots, m; j = 1, \dots, n \\
 & \sum_{k \in R_j} z_{ik} \leq n(1 - z_{ij}) \quad i = 1, \dots, m; j = 1, \dots, n \\
 & x_{ij} \geq 0, y_{ilj} \geq 0, v_{ij} \geq 0, w_{ij} \geq 0, z_{ij} \in \{0, 1\} \quad i, l = 1, \dots, m; j = 1, \dots, n
 \end{aligned}$$

Notar que este problema se puede separar en la parte continua y entera, tal que nos

quede como

$$\begin{aligned}
\theta_1(\lambda)_{\lambda \geq 0} &= \text{máx} \sum_{i=1}^m \sum_{j=1}^n \{p_j v_{ij} - x_{ij}(c_j + d_j + \lambda_{ij})\} \\
s.a. \quad r_j x_{ij} &= v_{ij} + w_{ij} + \sum_{l=1, l \neq i}^m y_{ilj} & i = 1, \dots, m; j = 1, \dots, n \\
\sum_{j=1}^n x_{ij} &\leq S_i & i = 1, \dots, m \\
w_{ij} + \sum_{l=1, l \neq i}^m y_{lij} &\geq f_{ij} & i = 1, \dots, m; j = 1, \dots, n \\
x_{ij} \geq 0, y_{ilj} \geq 0, v_{ij} \geq 0, w_{ij} \geq 0 & & i, l = 1, \dots, m; j = 1, \dots, n
\end{aligned}$$

Más aún, el problema se podría descomponer por índices. Notar que de esta manera queda un problema de programación lineal, con lo que se podría utilizar Simplex para poder resolverlo. Respecto al problema en las variables enteras, este corresponde a

$$\begin{aligned}
\theta_2(\lambda)_{\lambda \geq 0} &= \text{máx} \sum_{i=1}^m \sum_{j=1}^n z_{ij}(S_i \lambda_{ij} - F_j) \\
s.a. \quad \sum_{k \in R_j} z_{ik} &\leq n(1 - z_{ij}) & i = 1, \dots, m; j = 1, \dots, n \\
z_{ij} &\in \{0, 1\} & i = 1, \dots, m; j = 1, \dots, n
\end{aligned}$$

Notar que este es un problema de variables enteras, por lo que se podría resolver con Branch and Bound. Más aún, notar que tiene una estructura muy parecida a la de un problema de mochila, con lo que se podría también resolver con programación dinámica.

- b) Proponga una heurística lagrangiana para obtener una solución primal factible, basada en la Relajación Lagrangiana de la parte anterior.

Solución: Notar que la respuesta va a depender de que restriccion(es) hayan relajado en el inciso anterior. Para efectos de este inciso se utilizará la relajación de la pauta. Suponiendo que estamos en una iteración del algoritmo de subgradiente para relajación lagrangeana, es necesario transformar nuestra solución actual infactible, ya que podría no estar cumpliendo la restricción que relajamos, en una factible. Para ser más específicos, la restricción que podríamos estar incumpliendo es

$$x_{ij} \leq S_i z_{ij} \quad i = 1, \dots, m; j = 1, \dots, n$$

Con ello, si estamos en una iteración n , con variables primales $(x^n, y^n, v^n, w^n, z^n)$ y variable dual (λ^n) una posible heurística lagrangeana es:

-
1. Si (x^n, z^n) cumplen con la restricción (4), se tiene una solución factible.
 2. Si lo anterior no se cumple, es que tenemos que tenemos una superficie sembrada distinto de cero para un cierto producto cuando en realidad se decidió no sembrar
 3. Para solucionarlo, se puede fijar el x_{ij} correspondiente en valor 0
 4. Notar que al hacer esto estamos también alterando la restricción (1) y nos puede dar infactible ya que estamos modificando los flujos. Para solucionar lo anterior, lo que se puede hacer es alterar los flujos de tal manera que se cumpla la siguiente relación $\sum_{l=1; l \neq i}^m \{y_{lij} - y_{ilj}\} \geq f_{ij} + v_{ij}$
- c) Suponga que w_{45} es el valor del dual que se obtiene al relajar las restricciones (4) y (5), y w_4 es el valor del dual que se obtiene al relajar las restricciones (4) solamente. Sea z_0 el valor de la relajación lineal del problema, y z^* el valor óptimo del primal. Dibuje, en una recta, los valores de w_{45} , w_4 , z_0 y z^* . Justifique.

Solución: Tenemos

